Datapath 1

Aldo Mauricio Villegas Peralta, Uriel Lemus Pérez,
Max Ramos Martínez, Alfredo Ramírez Candia,
Santiago Silverio Flores

Benemérita Universidad Autónoma de Puebla, Puebla, México
vp202368617@alm.buap.mx, lp202362797@alm.buap.mx,
rm202376182@alm.buap.mx, rc202375947@alm.buap.mx,
sf202367190@alm.buap.mx

Resumen Se diseña y verifica por simulación un datapath de 8 bits formado por un multiplexor de entrada, una unidad aritmético-lógica con tres operaciones ($n+x$, $2n+x$ y $n+1$), un registro con realimentación y un buffer triestado. El sistema se describe en VHDL-2008 y se documenta su verificación mediante cinco bancos de prueba en GHDL y capturas de GTKWave. Los resultados registrados confirman la carga, la retención cuando $L = 0$, la alta impedancia de salida y el comportamiento aritmético módulo 2^8 dentro de la codificación propuesta.

Palabras clave: Arquitectura de computadoras · Datapath · VHDL · ALU · GHDL

1. Introducción

La unidad central de procesamiento de un computador se puede estudiar mediante dos subsistemas complementarios: la ruta de datos, o *datapath*, que transporta, opera y almacena operandos, y la unidad de control, que decide las operaciones y su secuencia. Esta separación se presenta en la literatura de arquitectura de computadores como una abstracción fundamental para comprender el diseño del hardware digital [14,6,7]. En una ruta de datos elemental, los multiplexores seleccionan las fuentes de información; la unidad aritmético-lógica (ALU) realiza transformaciones; y los registros proporcionan memoria de estado entre ciclos de reloj.

El lenguaje VHDL permite describir formalmente estas funciones sin depender de una realización física particular. Los bloques combinacionales y secuenciales pueden comprobarse de manera individual e integrada antes de considerar su implementación física [1,10,2]. En este trabajo se examina únicamente el **Datapath 1** del material de la asignatura Arquitectura de Computadoras. El esquema de referencia contiene un MUX, una ALU con operaciones aritméticas, un registro realimentado y una salida habilitada. El trabajo del equipo consistió en convertir ese esquema en una descripción VHDL-2008 y documentar los resultados de la simulación.

La fotografía original no permite identificar de forma inequívoca todos los códigos de control ni el ancho de los buses. Por esa razón se adoptó un **modelo**

explícito de implementación: ancho parametrizable, con valor inicial de ocho bits; tres códigos de operación; registro con reinicio síncrono; y palabra de control externa de seis bits. Estas decisiones se señalan como *propuestas del equipo*, no como especificaciones confirmadas del profesor. La distinción es importante porque una simulación correcta valida el modelo elegido, pero no resuelve por sí misma las ambigüedades del dibujo inicial.

Los objetivos son: describir los cuatro bloques mediante VHDL, integrarlos con la realimentación del registro, definir las palabras de control, documentar las pruebas sobre operaciones normales y casos límite, e interpretar las señales temporales obtenidas con GHDL y GTKWave. El alcance excluye una máquina de estados de control, la síntesis en hardware y los demás datapaths del repositorio. El documento desarrolla primero la arquitectura, después la propuesta experimental, las simulaciones y sus resultados; finalmente presenta conclusiones, fuentes bibliográficas y los cinco módulos de VHDL en un apéndice.
La elección de bloques sencillos permite estudiar por separado el recorrido del dato y las condiciones que autorizan su almacenamiento. Este enfoque resulta especialmente útil durante una práctica de arquitectura: si una operación aritmética devuelve el resultado esperado, todavía es necesario comprobar que la selección de entrada fue correcta, que el registro tomó ese resultado en el flanco previsto y que la salida visible corresponde al dato almacenado. Por ello, el análisis no se limita a las fórmulas de la ALU, sino que considera simultáneamente las señales de control y el estado anterior del circuito.

La realimentación añade un aspecto importante respecto de una suma puramente combinacional. El resultado de una operación puede constituir el operando de la siguiente, de manera que la interpretación de cada palabra de control depende del valor previamente guardado en Q. Una misma selección aritmética, por ejemplo el incremento, produce valores sucesivos diferentes si el registro se actualiza entre operaciones. En cambio, cuando $L = 0$ se conserva el valor anterior y se interrumpe esa evolución, aunque la ALU siga calculando un resultado de forma combinacional. Esta diferencia motiva las pruebas específicas de retención y de observación de señales internas.

Para acotar la práctica se mantiene una única ruta de datos, sin añadir memorias, contador de programa ni secuenciador. Las entradas de control se aplican desde el banco de prueba, lo que permite reproducir situaciones concretas y localizar con mayor precisión una posible discrepancia. Las decisiones que no pudieron verificarse en el esquema original, como la codificación binaria y la polaridad del MUX, permanecen explícitas para evitar confundir una elección de implementación con un requisito confirmado.

2. Diseño teórico

2.1. Fundamentos de la ruta de datos

El datapath es la parte ejecutora de un sistema digital: recibe entradas de datos y señales de selección, realiza microoperaciones y conserva resultados. En

un procesador completo, la unidad de control puede generar las señales de manera automática; aquí son aplicadas directamente por el banco de pruebas. Así se evita atribuir al circuito una FSM que no aparece en el diagrama proporcionado. Una FSM sería una posible ampliación, no un requisito de esta implementación [12,16,11].

La diferencia entre elementos combinacionales y secuenciales determina el comportamiento temporal. El MUX y la ALU responden a sus entradas sin esperar un flanco de reloj. En cambio, el registro cambia su contenido únicamente bajo las condiciones de reloj, carga y reinicio seleccionadas. El resultado de la ALU puede variar mientras la salida del registro permanece constante: esa separación es precisamente lo que comprueba una de las pruebas principales [18,9].

2.2. Arquitectura y recorrido de los datos

La Fig. 1 presenta el diagrama base utilizado en la práctica. La entrada `dato_ext` se conecta al MUX junto con una constante cero. La señal M elige el operando A; el otro operando, B, proviene de la salida Q del registro. Por tanto, el registro mantiene el valor previo n y permite efectuar operaciones sobre él. La ALU produce F_{ALU}, que alimenta directamente la entrada D del registro. La salida externa se toma de Q, pasando por un buffer que depende de $O.E.$.

Se distinguen cuatro señales conceptualmente diferentes: A es la salida del MUX; F_{ALU} es un resultado combinacional; $D = F_{\mathrm{ALU}}$ entra al registro; y Q es su valor almacenado. La salida externa es Q cuando está habilitada, o alta impedancia en caso contrario. Esta lógica queda expresada mediante

$$A = \begin{cases} \texttt{dato_ext}, & M = 1, \\ 0, & M = 0, \end{cases} \qquad B = Q, \qquad D = F_{\mathrm{ALU}} = \mathrm{ALU}(A, B, C). \tag{1}$$

Para cada flanco ascendente del reloj, la ecuación secuencial adoptada es

$$Q^{+} = \begin{cases} 0, & \texttt{rst} = 1, \\ D, & \texttt{rst} = 0 \text{ y } L = 1, \\ Q, & \text{en otro caso.} \end{cases} \tag{2}$$

Finalmente, la habilitación de salida se define como

$$\texttt{salida} = \begin{cases} Q, & O.E. = 1, \\ Z, & O.E. = 0. \end{cases} \tag{3}$$

En particular, con $L = 0$ puede cambiar F_{ALU}, pero Q se mantiene sin alteración. El reinicio síncrono se añadió para garantizar un estado conocido al comienzo de las simulaciones; no está indicado explícitamente en la fotografía original.

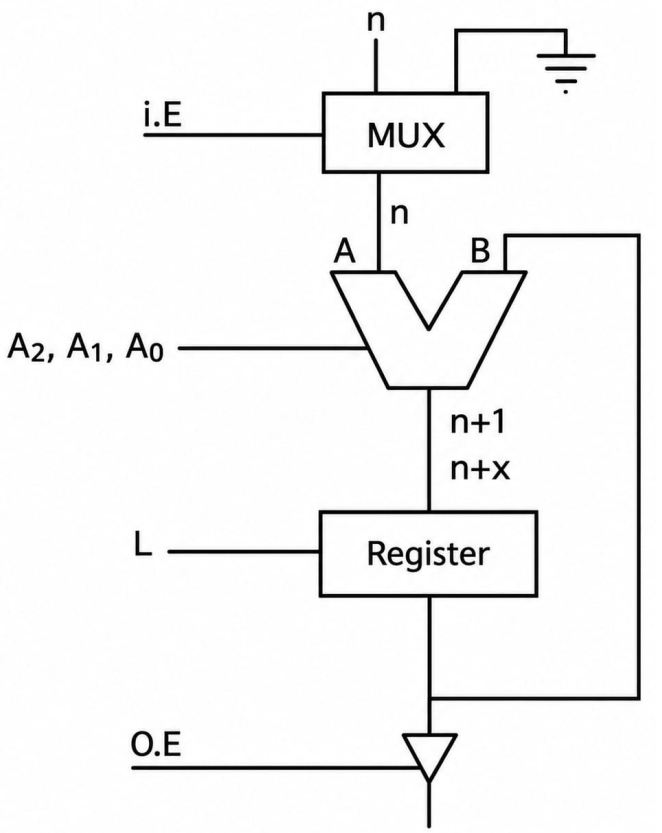


Fig. 1. Diagrama del Datapath 1 proporcionado para la práctica. Se observa el MUX de entrada, la unidad aritmética, el registro con carga L, la realimentación de Q hacia la entrada B y el buffer de salida controlado por $O.E.$.

2.3. Descripción de los módulos

El **multiplexor de entrada** selecciona entre el dato externo y la constante cero. Su control M posee polaridad propuesta: con $M = 1$ se selecciona el dato y con $M = 0$ se selecciona cero. Al tratarse de una operación combinacional, la salida sigue inmediatamente la selección aplicada.

La **ALU** recibe A y B como vectores sin signo y un selector C de tres bits. Implementa tres operaciones conforme a la Tabla 2. Para $2n + x$ se desplaza B una posición a la izquierda dentro de la ALU; esto no requiere un módulo de desplazamiento separado. Un *barrel shifter* genérico permitiría desplazamientos variables, pero no forma parte del esquema estudiado [15].

El **registro** conserva el estado y dispone de habilitación de carga L. El reinicio síncrono tiene prioridad sobre L, como especifica la Ec. (2). El **buffer de salida** propaga Q si está habilitado y asigna el valor lógico `'Z'` a la salida en otro caso, de acuerdo con las capacidades de la lógica multivaluada de `std_logic` [9].

Tabla 1. Formato propuesto de la palabra de control. La correspondencia exacta entre bits y señales no pudo verificarse en el dibujo original.

Bits	Campo	Función
$CW[5]$	$O.E.$	Habilita la salida externa desde Q.
$CW[4]$	L	Carga el registro en un flanco ascendente.
$CW[3]$	M	Selecciona A: 1 = dato externo; 0 = cero.
$CW[2:0]$	C	Selecciona la operación aritmética.

Tabla 2. Operaciones de ALU y códigos *propuestos*, no confirmados por el profesor.

C	Operación	Resultado de la ALU	Expresión
`000`	Suma	$F = A + B$	$n + x$
`001`	Doble más suma	$F = (B \ll 1) + A$	$2n + x$
`010`	Incremento	$F = B + 1$	$n + 1$
`011--111`	Reserva	$F = 0$ (valor provisional)	—

2.4. Palabra de control y microoperaciones

En este trabajo una palabra de control de seis bits se interpreta como $CW = [O.E., L, M, C_2, C_1, C_0]$. Los tres bits menos significativos escogen la operación de la ALU; los tres restantes activan la salida, la carga y el MUX. Los códigos de la ALU son una convención de implementación: el esquema del profesor deja entrever las expresiones matemáticas, pero no permite leer de manera confiable los códigos binarios.

Al aplicar `111000`, la ALU suma el contenido previo del registro con la entrada seleccionada, carga el resultado y lo muestra después del flanco de reloj. La palabra `111001` calcula $2n + x$ y lo registra; `110010` incrementa n; y `100010` calcula $n + 1$ sin almacenarlo porque $L = 0$. Esta última combinación separa la actividad combinacional de la secuencial. La carga inicial de un dato x puede obtenerse tras el reinicio: entonces $Q = 0$, y la suma $Q + x$ da x.

Tabla 3. Ejemplo de microoperaciones documentadas en el banco de pruebas integrado (valores hexadecimales después del flanco pertinente).

CW	F_{ALU}	Q^+	Salida	**Interpretación**
`111000`	`0x05`	`0x05`	`0x05`	Suma, carga y salida.
`111001`	`0x0F`	`0x0F`	`0x0F`	Doble más suma, carga.
`100010`	`0x10`	`0x0F`	`0x0F`	$L = 0$: conserva Q.
`100000`	`0x0F`	`0x0F`	`0x0F`	Conservación y lectura.
`011000`	`0x3C`	`0x3C`	Z	Carga con salida deshabilitada.
`110010`	`0x3D`	`0x3D`	`0x3D`	Incremento posterior.

2.5. Aritmética, ancho parametrizable y limitaciones

El genérico `ANCHO` se utiliza para definir la longitud de los buses del MUX, ALU, registro y buffer. Se toma `ANCHO=8` como referencia, por lo que los valores sin signo están entre 0 y 255. Como no se incorpora una bandera de acarreo, toda operación se realiza módulo 2^{ANCHO}. Para ocho bits, $300 \bmod 256 = 44$ (`0x2C`); $400 \bmod 256 = 144$ (`0x90`); y $256 \bmod 256 = 0$ (`0x00`). Por tanto, el resultado matemático exacto no siempre cabe en el ancho configurado.

La representación de $2n + x$ requiere también tener presente que el desplazamiento lógico descarta los bits que exceden la anchura disponible. Para evitar desbordamiento en ocho bits se necesita $2n + x \leq 255$. El diseño no posee banderas de cero ni de acarreo. Tampoco admite una carga arbitraria de A cuando $Q \neq 0$ con las únicas tres operaciones seleccionadas: la carga directa documentada requiere colocar primero $Q = 0$. Todas estas son limitaciones del modelo elegido, no fallos de las simulaciones.

3. Propuesta experimental

La verificación documentada siguió la secuencia: definir las funciones de cada módulo; describir las entidades en VHDL-2008; integrar los bloques; construir bancos de pruebas autocomprobables; y observar las transiciones temporales. Los resultados del informe de origen proceden de GHDL con el estándar `--std=08` y de GTKWave para interpretar formas de onda [4,5]. La herramienta de simulación generó archivos VCD que se emplearon para las capturas incluidas aquí.

Se describen cinco bancos de prueba: `tb_mux_entrada`, `tb_registro`, `tb_buffer_salida`, `tb_alu` y `tb_datapath`. El primero evalúa ambas posiciones del selector del MUX y anchos de ocho y cuatro bits; el segundo, el reinicio, la carga y la conservación del estado; el tercero, el comportamiento del buffer y de la alta impedancia; el cuarto, las operaciones de la ALU y los casos de desbordamiento; y el quinto, las palabras de control y la realimentación. Cada banco compara los valores reales frente a los esperados mediante aserciones. Los tiempos de finalización reportados son estímulos de simulación, no medidas de rendimiento físico del circuito.

Como procedimiento reproducible para el repositorio se documentan las órdenes `scripts/vhdl-analizar datapath-1` y `scripts/vhdl-sim datapath-1 <tb>`, donde `<tb>` se sustituye por el banco elegido; la generación del reporte utiliza el script `latexbuild`. Las capturas incluidas se preservaron de los registros gráficos de la práctica, sin alterar los valores representados. El criterio de éxito del experimento es la terminación de los cinco bancos sin aserciones fallidas.

Además de comprobar el resultado final de cada operación, el procedimiento experimental distingue tres niveles de observación: la salida combinacional F_{ALU}, el estado interno Q y la salida habilitada hacia el exterior. Esta separación permite identificar si una discrepancia corresponde a la selección de operandos, al instante de carga del registro o a la habilitación del buffer. Observar únicamente el puerto de salida no sería suficiente, porque un valor almacenado correctamente podría permanecer oculto durante un intervalo con $O.E. = 0$.

Los estímulos se organizaron para cubrir situaciones normales y casos límite. Las primeras verificaciones confirman la elección del MUX, la carga y el reinicio del registro, y la propagación o desconexión del buffer. Después se comprueban las tres funciones implementadas en la ALU, incluidos operandos cero, valores elevados y resultados que exceden la anchura del bus. Por último, la simulación integrada combina controles para comprobar que las conexiones entre módulos reproducen el comportamiento de las pruebas individuales. Se documentan también instancias de cuatro bits, que ayudan a distinguir una operación genérica de una implementación fija de ocho bits.

El análisis de las capturas se realiza contrastando intervalos temporales con la palabra de control activa y con los valores conocidos antes y después de los flancos. En particular, la prueba de $L = 0$ exige comparar el cálculo nuevo de la ALU con el estado retenido, mientras que la prueba de $O.E. = 0$ exige identificar la alta impedancia sin atribuirla a una pérdida del contenido del registro. Las formas de onda se usan, por tanto, como evidencia temporal complementaria a las aserciones del banco, y no como sustituto de una verificación automática de valores.

Como criterio de aceptación se utiliza la terminación satisfactoria de los cinco bancos descritos. Sus instantes de finalización indican hasta qué punto se aplicaron estímulos en cada simulación; no miden la velocidad de un circuito sintetizado. De igual modo, los resultados de la aritmética modular se comparan con el valor esperado para el ancho configurado, no con el resultado entero

ilimitado. Esta precaución evita clasificar como fallos operaciones cuya pérdida de bits está especificada en el propio modelo.

4. Simulaciones en VHDL

La Fig. 2 muestra un intervalo de la simulación integrada. Tras el reinicio, la suma con $CW = \texttt{111000}$ actualiza el registro a `0x05`; la operación $2n + x$ con $CW = \texttt{111001}$ lo lleva a `0x0F`. Posteriormente, con $CW = \texttt{100010}$, la ALU produce `0x10`, pero el registro sigue reteniendo `0x0F` porque $L = 0$. Esta comparación es la evidencia fundamental de que la salida externa procede del registro y no directamente de la ALU.

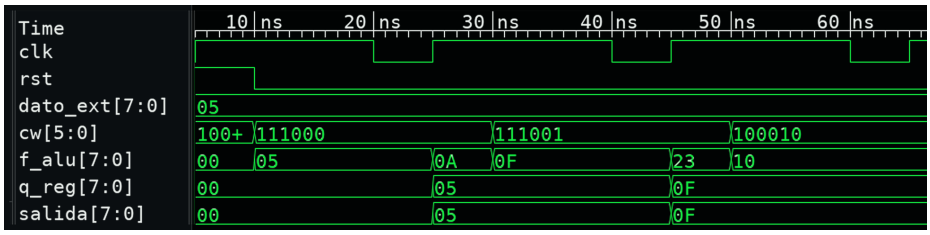


Fig. 2. Ondas reales incluidas en el documento original: secuencia de suma, doble más suma y retención del registro ($L = 0$). Se conserva el eje temporal y las señales originales.

La Fig. 3 recoge otro intervalo de la misma simulación. La palabra `011000` permite cargar `0x3C` mientras el buffer permanece en alta impedancia; después, `100000` muestra el dato almacenado. El incremento con `110010` actualiza el estado a `0x3D`. La salida permanece así desacoplada de la actividad de la ALU cuando $O.E. = 0$.

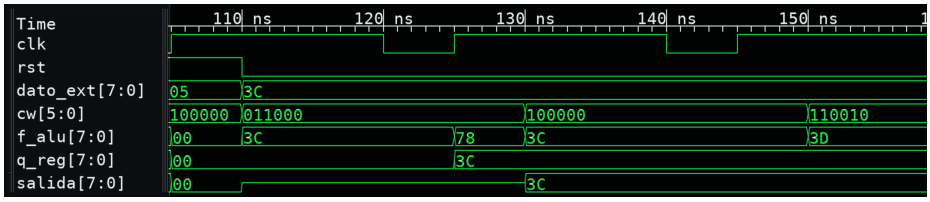


Fig. 3. Ondas reales del datapath integrado en la prueba de carga, alta impedancia, lectura y posterior incremento.

El banco de la ALU combina operaciones de ocho y cuatro bits. La Fig. 4 presenta los resultados con ancho de ocho bits, incluidos casos de desbordamiento.

La Fig. 5 muestra los estímulos de cuatro bits, que permiten comprobar la parametrización del ancho. Las señales corresponden a los registros visuales obtenidos durante la simulación; sus valores no fueron reconstruidos ni interpolados.

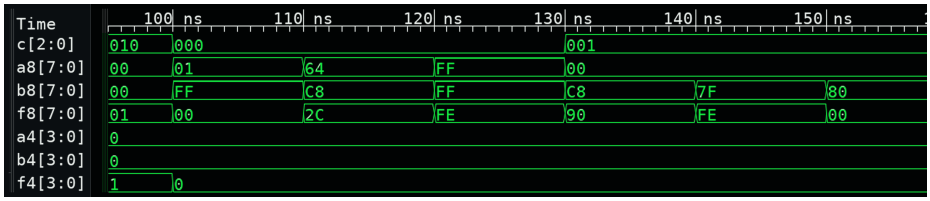


Fig. 4. Formas de onda originales de la ALU de ocho bits; se observan operandos, selector y resultado, incluidos desbordamientos modulares.

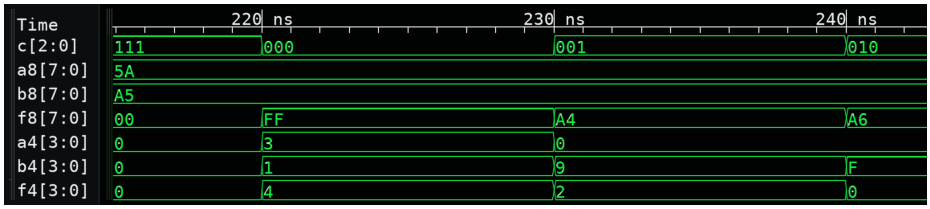


Fig. 5. Formas de onda originales de la instancia de cuatro bits: reutilización de la ALU con el genérico `ANCHO=4`.

5. Resultados

5.1. Ejecución de las pruebas documentadas

La Tabla 4 resume los tiempos de terminación y los resultados registrados para cada banco de pruebas. Los cinco bancos aparecen registrados como satisfactorios. Es importante distinguir la coincidencia de sus aserciones en simulación de una demostración de sintetizabilidad: estos bancos verifican las funciones descritas, pero no proporcionan información acerca de recursos físicos, tiempos de propagación reales, potencia o frecuencia máxima.

5.2. Operaciones normales y desbordamientos

La Tabla 5 utiliza una muestra de los estímulos documentados. Con $n = 5$ y $x = 10$, la suma produce $15 = \texttt{0x0F}$, mientras que $2n + x$ produce $20 = \texttt{0x14}$. El incremento de $n = 5$ da $6 = \texttt{0x06}$, independientemente del valor de A. Estos

Tabla 4. Resultados registrados durante la ejecución de los cinco bancos de prueba.

Banco de prueba	Tiempo final	Resultado
`tb_mux_entrada`	60 ns	OK
`tb_registro`	50 ns	OK
`tb_buffer_salida`	40 ns	OK
`tb_alu`	250 ns	OK (27 comprobaciones)
`tb_datapath`	170 ns	OK

Tabla 5. Resultados aritméticos correspondientes al banco `tb_alu`.

C	A	B	F	Cálculo o caso
000	0A	05	0F	$5 + 10 = 15$
000	3C	00	3C	Carga con $B = 0$
001	0A	05	14	$2(5) + 10 = 20$
001	00	7F	FE	$2(127) = 254$
010	A5	05	06	$5 + 1 = 6$; ignora A
000	64	C8	2C	$300 \bmod 256 = 44$
001	00	C8	90	$400 \bmod 256 = 144$
010	00	FF	00	$256 \bmod 256 = 0$

tres ejemplos verifican las expresiones que se atribuyen al dibujo del curso, bajo la asignación $A = x$ y $B = n$ que se decidió implementar.

En el caso de ocho bits, los resultados `0x2C`, `0x90` y `0x00` no son errores aritméticos de la implementación: son los residuos de la operación dentro del intervalo representable de 0 a 255. El significado de una operación que sobrepasa ese intervalo depende del contexto en que se use el datapath; aquí se ha optado por la aritmética modular sin indicadores adicionales. Este aspecto debe diferenciarse de la saturación, que limitaría el resultado a 255, y del uso de un bus más ancho, que conservaría los bits altos.

La instancia de cuatro bits reproduce el comportamiento sobre otro módulo: $3 + 1 = 4$; $2(9) = 18 \equiv 2 \pmod{16}$; y $15 + 1 \equiv 0 \pmod{16}$. Esto demuestra que el ancho no está fijado por constantes numéricas dentro de las operaciones, sino por el genérico aplicado a los vectores de entrada y salida.

5.3. Respuesta del registro y del buffer

La Tabla 3 resume la separación entre F_{ALU}, Q y la salida externa. La fila `100010` es especialmente relevante: $F_{\mathrm{ALU}} =$ `0x10` mientras $Q =$ `0x0F` y la salida sigue mostrando `0x0F`. En la fila `011000`, la ALU y el registro contienen `0x3C`, pero el buffer muestra alta impedancia. En ambos casos, el comportamiento observado corresponde a las ecuaciones del modelo, no a una conexión directa entre el resultado de la ALU y el exterior.

Temporalmente, la ALU ofrece un resultado combinacional antes del flanco de captura, y el registro solo cambia cuando coincide el flanco con la habilitación

de carga. Esta distinción aparece en las formas de onda y sirve para verificar las conexiones de la Fig. 1. También aclara el significado de las palabras de control: los campos del MUX y de la ALU pueden provocar cambios en señales internas aunque L u $O.E.$ impidan que la salida externa cambie.

5.4. Lectura ciclo a ciclo de la secuencia integrada

Las palabras de control también permiten reconstruir un ejemplo completo sin depender de las imágenes. Si se parte de un registro reiniciado, $Q_0 = 0$, y se selecciona el dato externo $x = 5$ con $CW =$ `111000`, la ALU obtiene $F_{\mathrm{ALU}} = 0+5 = 5$; en el siguiente flanco, el registro almacena $Q_1 = 5$. Con $x = 5$ y $CW =$ `111001`, la ALU evalúa $2(5) + 5 = 15$, y la carga habilitada produce $Q_2 = 15$. En el tercer intervalo, $CW =$ `100010` solicita la operación $15 + 1 = 16$; no obstante, al estar $L = 0$, el estado después del flanco permanece en $Q_3 = 15$. Se trata de tres configuraciones consecutivas del mismo hardware, no de tres circuitos diferentes.

La secuencia posterior que incluye `0x3C` ofrece una comprobación complementaria. Tras el reinicio, $CW =$ `011000` carga el valor externo 60 (`0x3C`), pero la salida permanece en alta impedancia porque $O.E. = 0$. Cambiar a $CW =$ `100000` no altera Q: únicamente permite leer el registro. Por último, $CW =$ `110010` calcula $60 + 1 = 61$ (`0x3D`), valor que se guarda con $L = 1$. Estos ejemplos distinguen tres efectos independientes: selección aritmética, escritura del estado y visibilidad externa.

La tabla de microoperaciones tiene utilidad más allá de enumerar códigos. Puede emplearse como referencia para diagnosticar posibles errores de cableado: una salida que adoptara `0x10` durante la prueba $L = 0$ indicaría que el buffer está recibiendo la ALU en lugar del registro; una salida no definida cuando $O.E. = 1$ sugeriría una conexión distinta a Q; y un registro que cargara con $L = 0$ contradiría la ecuación secuencial. Estos son criterios de interpretación deducidos de las señales descritas, no incidencias que se afirme haber observado en las pruebas.

5.5. Alcance de la evidencia

Los resultados anteriores son suficientes para discutir las funciones simuladas, pero no permiten establecer prestaciones temporales o de consumo eléctrico. No se realizaron ni documentaron pruebas de síntesis o implementación física. GHDL dispone de un flujo de síntesis experimental, pero no fue utilizado para la evaluación presentada [4]. Por consiguiente, los tiempos del banco de prueba son marcas de simulación, no mediciones de latencia física.

Del mismo modo, la codificación de la ALU y la polaridad adoptada para M son decisiones del equipo. Si el profesor entregara posteriormente una tabla de control diferente, sería necesario adecuar tanto los estímulos como la descripción del circuito y repetir las verificaciones. Estas limitaciones no invalidan la coherencia interna de las pruebas registradas; indican el alcance exacto de lo demostrado.

La comparación de los bancos individuales y de la integración permite valorar también la cobertura alcanzada. Los casos de ALU establecen que las tres expresiones seleccionadas dan resultados coherentes para entradas concretas; el banco integrado muestra que esos resultados llegan al registro mediante las conexiones previstas y que la salida externa respeta las señales de carga y de habilitación. Se trata de propiedades complementarias: la corrección aislada de la ALU no demostraría por sí sola que B esté conectado a Q o que el buffer reciba el dato almacenado.

Los ensayos con anchos distintos aportan una comprobación adicional de parametrización. Si un bloque dependiera inadvertidamente de una constante fija de ocho bits, al ejecutarlo con cuatro bits podrían aparecer discrepancias de rango, truncamientos distintos o asignaciones incompatibles. Los resultados documentados muestran el comportamiento modular esperado en esos ejemplos. Sin embargo, no se afirma que todos los valores posibles de entrada ni todas las combinaciones de control hayan sido explorados exhaustivamente; las pruebas aportan evidencia sobre los escenarios incluidos en los bancos.

5.6. Consistencia entre ecuaciones, tablas y ondas

Las ecuaciones de diseño, la tabla de selección de ALU y la tabla de palabras de control se refieren a los mismos operandos: A procede del MUX y B procede de Q. Este acuerdo evita una ambigüedad frecuente en el análisis de rutas con realimentación: confundir el dato nuevo de entrada con el dato almacenado del ciclo anterior. La coherencia numérica entre los ejemplos $5 + 10 = 15$, $2(5) + 10 = 20$ y $5 + 1 = 6$ confirma que las expresiones de la tabla de operaciones corresponden a la asignación adoptada.

La comparación entre los anchos de cuatro y ocho bits muestra, además, que el desbordamiento no depende únicamente de la operación, sino del tamaño del registro que recibe el resultado. El mismo incremento produce 16 cuando existe capacidad suficiente, pero produce 0 en un vector de cuatro bits al calcular $15+1$. La ALU no realiza una operación diferente: cambian los límites de representación. Al presentar estos casos, es esencial conservar tanto el resultado modular como la expresión matemática completa, porque omitir cualquiera de los dos podría hacer parecer errónea una simulación correcta.

6. Conclusiones

Se documentó el diseño y la verificación por simulación de un datapath de ocho bits, con ancho parametrizable, compuesto por MUX, ALU, registro con carga y buffer triestado. La realimentación de Q hacia la ALU permite ejecutar operaciones sucesivas sobre el contenido almacenado. El papel de cada bloque se explica mediante ecuaciones, tablas de control y señales temporales.

El resultado principal es la diferenciación entre las señales combinacionales y el estado registrado: la ALU puede producir una salida nueva mientras que Q conserva su valor si $L = 0$. Las formas de onda y la tabla de microoperaciones

también documentan que el buffer puede desconectar la salida mediante alta impedancia, sin borrar el estado almacenado. La ALU desarrolla las tres operaciones $n + x$, $2n + x$ y $n + 1$, y sus resultados siguen una aritmética modular de ancho fijo. Los casos de desbordamiento son coherentes con esa definición.

El diseño permanece limitado a una ruta de datos gobernada externamente, sin banderas y sin una unidad de control automática. La adición de una FSM permitiría secuenciar microoperaciones, pero por sí sola no constituiría un procesador completo: aún serían necesarios bloques y mecanismos relativos a instrucciones y control de ejecución. Como trabajo futuro se plantea contrastar la codificación definitiva del profesor, repetir los bancos de prueba después de cualquier modificación y evaluar, si procede, una implementación sintetizable. Las conclusiones se refieren a los resultados de simulación documentados, no a pruebas de hardware.

La práctica pone de manifiesto que una arquitectura aparentemente simple requiere especificar con claridad sus estados y controles. El MUX determina el dato que participa en la operación, la ALU transforma operandos sin almacenar información y el registro fija cuándo un resultado pasa a formar parte del estado. El buffer, por su parte, controla la visibilidad externa sin modificar ese estado. La observación independiente de estos efectos permite interpretar de forma más precisa las transiciones que aparecen en las capturas y explica por qué no basta con comprobar exclusivamente el resultado aritmético.

También se identificó la importancia de documentar las restricciones de representación. Al trabajar con ocho bits, las operaciones que rebasan 255 conservan únicamente el resultado modular y no producen una bandera que anuncie la pérdida de información. Esta característica es válida para el modelo construido, pero condiciona los usos posteriores del circuito. Del mismo modo, el reinicio síncrono facilita la preparación de un estado conocido para las pruebas, aunque representa una decisión de diseño adicional respecto del dibujo del curso. Ambas condiciones deben considerarse antes de integrar el datapath en un sistema más amplio.

En términos de verificación, los cinco bancos de prueba ofrecen un procedimiento repetible para confirmar las funciones descritas y detectar regresiones cuando se modifiquen los módulos. Los resultados registrados son consistentes con la arquitectura propuesta, pero su alcance permanece limitado a simulación funcional: no se obtuvieron métricas de área, consumo, frecuencia máxima ni comportamiento físico. La siguiente etapa razonable sería contrastar la tabla de control con una especificación oficial legible y, solo después, ampliar las operaciones o avanzar hacia una implementación física. Así se conservaría la trazabilidad entre el diseño solicitado, su descripción VHDL y la evidencia experimental.

Referencias

1. Ashenden, P.J.: *The Designer's Guide to VHDL*. 3.ª ed., Morgan Kaufmann (2008). ISBN 978-0-12-088785-9.
2. Brown, S., Vranesic, Z.: *Fundamentals of Digital Logic with VHDL Design*. 3.ª ed., McGraw-Hill (2009). ISBN 978-0-07-352953-0.

3. Bryant, R., O'Hallaron, D.: *Computer Systems: A Programmer's Perspective*. 3.ª ed., Pearson (2015). ISBN 978-0-13-409266-9.
4. GHDL developers: *GHDL: the VHDL simulator*. Documentación del proyecto: `https://ghdl.github.io/ghdl/`.
5. GTKWave developers: *GTKWave analyzer*. Documentación: `https://gtkwave.sourceforge.net/`.
6. Hamacher, V., Vranesic, Z., Zaky, S.: *Computer Organization*. 5.ª ed., McGraw-Hill (2002). ISBN 978-0-07-232086-2.
7. Harris, S., Harris, D.: *Digital Design and Computer Architecture: ARM Edition*. Morgan Kaufmann (2015). ISBN 978-0-12-800056-4.
8. Hennessy, J., Patterson, D.: *Computer Architecture: A Quantitative Approach*. 6.ª ed., Morgan Kaufmann (2017). ISBN 978-0-12-811905-1.
9. IEEE: *IEEE Standard Multivalue Logic System for VHDL Model Interoperability*. IEEE Std 1164-1993. DOI: 10.1109/IEEESTD.1993.115571.
10. IEEE: *IEEE Standard VHDL Language Reference Manual*. IEEE Std 1076-2008. DOI: 10.1109/IEEESTD.2011.5967868.
11. Katz, R., Borriello, G.: *Contemporary Logic Design*. 2.ª ed., Pearson (2005). ISBN 978-0-201-30857-0.
12. Mano, M., Kime, C., Martin, T.: *Logic and Computer Design Fundamentals*. 5.ª ed., Pearson (2016). ISBN 978-0-13-376063-7.
13. Navabi, Z.: *VHDL: Analysis and Modeling of Digital Systems*. 2.ª ed., McGraw-Hill (1998). ISBN 978-0-07-046479-7.
14. Patterson, D., Hennessy, J.: *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*. 1.ª ed. de la versión RISC-V, Morgan Kaufmann (2017). ISBN 978-0-12-812275-4.
15. Rabaey, J., Chandrakasan, A., Nikolić, B.: *Digital Integrated Circuits: A Design Perspective*. 2.ª ed., Pearson (2003). ISBN 978-0-13-090996-1.
16. Roth, C., Kinney, L.: *Fundamentals of Logic Design*. 7.ª ed., Cengage (2013). ISBN 978-1-133-62847-7.
17. Stallings, W.: *Computer Organization and Architecture: Designing for Performance*. 10.ª ed., Pearson (2015). ISBN 978-0-13-410161-3.
18. Wakerly, J.: *Digital Design: Principles and Practices*. 4.ª ed., Pearson (2005). ISBN 978-0-13-186389-7.

A. Código VHDL

Se incluyen las cinco entidades del Datapath 1, presentadas con indentación uniforme y cortes de línea en límites sintácticos. Se preservan los puertos, condiciones y operaciones de la implementación descrita en el cuerpo del trabajo. Los bancos de prueba se describen en la Sección 3; este apéndice reúne los módulos funcionales.

Listado 1.1. Multiplexor de entrada: módulo completo

```
-- Datapath 1: multiplexor 2 a 1.
-- sel='0' selecciona cero; sel='1' dato externo.
library ieee;
use ieee.std_logic_1164.all;

entity mux_entrada is
  generic (ANCHO : natural := 8);
  port (
    dato_ext : in std_logic_vector(
      ANCHO - 1 downto 0);
    sel : in std_logic;
    y : out std_logic_vector(
      ANCHO - 1 downto 0)
  );
end entity;

architecture comportamental of mux_entrada is
begin
  y <= dato_ext when sel = '1'
       else (others => '0');
end architecture;
```

Listado 1.2. Buffer triestado: módulo completo

```
-- Datapath 1: buffer triestado de salida.
-- oe='1' muestra Q; oe='0' entrega impedancia Z.
library ieee;
use ieee.std_logic_1164.all;

entity buffer_salida is
  generic (ANCHO : natural := 8);
  port (
    d_in : in std_logic_vector(ANCHO - 1 downto 0);
    oe : in std_logic;
    y : out std_logic_vector(ANCHO - 1 downto 0)
  );
end entity;

architecture comportamental of buffer_salida is
begin
  y <= d_in when oe = '1'
       else (others => 'Z');
end architecture;
```

Listado 1.3. ALU combinacional: módulo completo

```
-- Datapath 1: ALU combinacional.
-- Codigos propuestos; no son originales confirmados.
-- C=000: A+B; C=001: (B<<1)+A; C=010: B+1.
-- C=011..111: resultado cero (reserva provisional).
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity alu is
  generic (ANCHO : natural := 8);
  port (
    a : in std_logic_vector(ANCHO - 1 downto 0);
    b : in std_logic_vector(ANCHO - 1 downto 0);
    c : in std_logic_vector(2 downto 0);
    f : out std_logic_vector(ANCHO - 1 downto 0)
  );
end entity;

architecture comportamental of alu is
begin
  process (a, b, c)
  begin
    case c is
      when "000" =>
        f <= std_logic_vector(
          unsigned(a) + unsigned(b));
      when "001" =>
        f <= std_logic_vector(
          shift_left(unsigned(b), 1)
          + unsigned(a));
      when "010" =>
        f <= std_logic_vector(unsigned(b) + 1);
      when others =>
        f <= (others => '0');
    end case;
  end process;
end architecture;
```

Listado 1.4. Registro síncrono: módulo completo

```
-- Datapath 1: registro con carga habilitada.
-- Reinicio sincrono con prioridad sobre carga.
library ieee;
use ieee.std_logic_1164.all;

entity registro is
  generic (ANCHO : natural := 8);
  port (
    clk : in std_logic;
    rst : in std_logic;
    l : in std_logic;
    d : in std_logic_vector(ANCHO - 1 downto 0);
    q : out std_logic_vector(ANCHO - 1 downto 0)
  );
end entity;

architecture comportamental of registro is
  signal contenido : std_logic_vector(
    ANCHO - 1 downto 0) := (others => '0');
begin
  process (clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        contenido <= (others => '0');
      elsif l = '1' then
        contenido <= d;
      end if;
    end if;
  end process;
  q <= contenido;
end architecture;
```

Datapath integrado. Para facilitar su lectura, el módulo superior se presenta en tres fragmentos consecutivos, separados únicamente en límites de declaraciones e instancias. El archivo `src/datapath.vhd` adjunto contiene la unidad completa sin fragmentación.

Listado 1.5. Datapath integrado, parte 1 de 3: entidad, MUX y ALU

```
-- Datapath 1: interconexion de cuatro modulos.
-- CW=[OE,L,M,C2,C1,C0], codificacion provisional.
library ieee;
use ieee.std_logic_1164.all;

entity datapath is
  generic (ANCHO : natural := 8);
  port (
    clk : in std_logic;
    rst : in std_logic;
    dato_ext : in std_logic_vector(
      ANCHO - 1 downto 0);
    cw : in std_logic_vector(5 downto 0);
    salida : out std_logic_vector(
      ANCHO - 1 downto 0);
    f_alu : out std_logic_vector(
      ANCHO - 1 downto 0);
    q_reg : out std_logic_vector(
      ANCHO - 1 downto 0)
  );
end entity;

architecture estructural of datapath is
  component mux_entrada is
    generic (ANCHO : natural := 8);
    port (
      dato_ext : in std_logic_vector(
        ANCHO - 1 downto 0);
      sel : in std_logic;
      y : out std_logic_vector(
        ANCHO - 1 downto 0)
    );
  end component;
  component alu is
    generic (ANCHO : natural := 8);
    port (
      a : in std_logic_vector(ANCHO - 1 downto 0);
      b : in std_logic_vector(ANCHO - 1 downto 0);
      c : in std_logic_vector(2 downto 0);
      f : out std_logic_vector(ANCHO - 1 downto 0)
    );
  end component;
```

Listado 1.6. Datapath integrado, parte 2 de 3: componentes, señales y control

```
  component registro is
    generic (ANCHO : natural := 8);
    port (
      clk : in std_logic;
      rst : in std_logic;
      l : in std_logic;
      d : in std_logic_vector(ANCHO - 1 downto 0);
      q : out std_logic_vector(ANCHO - 1 downto 0)
    );
  end component;
  component buffer_salida is
    generic (ANCHO : natural := 8);
    port (
      d_in : in std_logic_vector(
        ANCHO - 1 downto 0);
      oe : in std_logic;
      y : out std_logic_vector(
        ANCHO - 1 downto 0)
    );
  end component;

  signal oe, l, m : std_logic;
  signal c : std_logic_vector(2 downto 0);
  signal a_bus, b_bus, f_bus, q_bus :
    std_logic_vector(ANCHO - 1 downto 0);
begin
  oe <= cw(5);
  l <= cw(4);
  m <= cw(3);
  c <= cw(2 downto 0);
  b_bus <= q_bus;
```

Listado 1.7. Datapath integrado, parte 3 de 3: conexiones y salidas

```
  u_mux : mux_entrada
    generic map (ANCHO => ANCHO)
    port map (
      dato_ext => dato_ext,
      sel => m,
      y => a_bus
    );

  u_alu : alu
    generic map (ANCHO => ANCHO)
    port map (
      a => a_bus, b => b_bus,
      c => c, f => f_bus
    );

  u_registro : registro
    generic map (ANCHO => ANCHO)
    port map (
      clk => clk, rst => rst, l => l,
      d => f_bus, q => q_bus
    );

  u_buffer : buffer_salida
    generic map (ANCHO => ANCHO)
    port map (
      d_in => q_bus, oe => oe,
      y => salida
    );

  f_alu <= f_bus;
  q_reg <= q_bus;
end architecture;
```